The University of British Columbia ORCA

HEART 2026

TinyRVV: A Minimalist Vector Subset for RISC-V

Jerry Zizhuo Yun zizhuoyun@gmail.com Guy Lemieux lemieux@ece.ubc.ca

Motivation:

Full Vector Support Is Expensive

- Embedded kernels repeat simple operations over arrays.
- Scalar code spends cycles on loop and address updates.
- Full RVV reduces overhead but adds significant complexity.
- Small cores need acceleration without large vector hardware.

Design Goal:

Minimal Useful Vector Support

- Start from benchmark needs, not full RVV coverage.
- Support only 32-bit integer vector operations.
- Keep vector state compact with VLEN=256.
- Leave room to add low-cost instructions later.

Full RVV-style core

Flexible but complex
Parallel vector lanes
Large hardware cost

Keep only what is needed

TinyRVV

Fixed 32-bit vectors
One element at a time
Reuses scalar ALU
Separate vector registers

Workload-Driven ISA Subset

Vector config	vsetvl, vsetvli, vsetivli
Memory	vle32.v, vse32.v
Arithmetic	vadd.vv, vadd.vx, vmul.vx
Logic	vand.vx, vand.vi
Shift	vsrl.vi

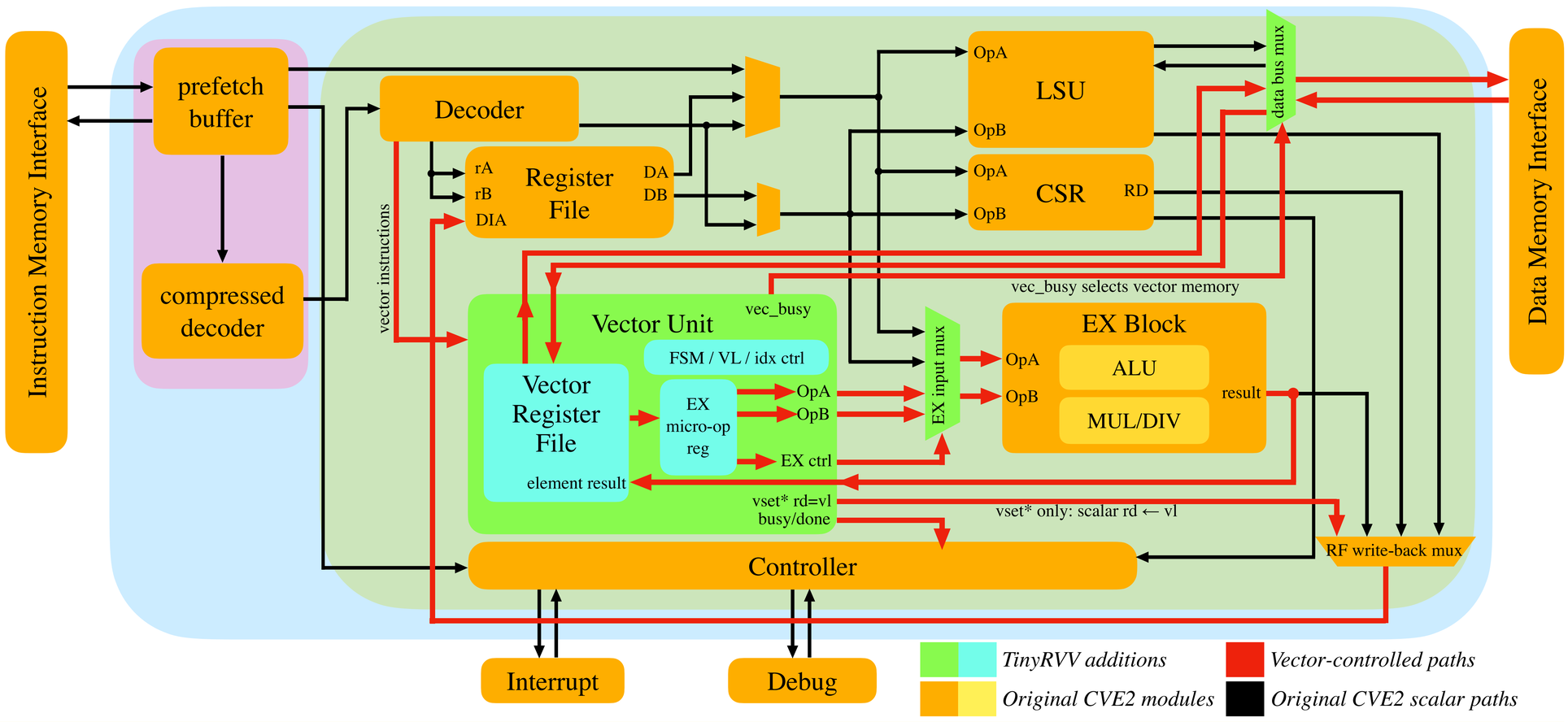
Minimal subset selected from benchmark requirements.

Useful vector acceleration does not require full RVV complexity or SIMD lanes!

Architecture:

Vector Side Path in CVE2

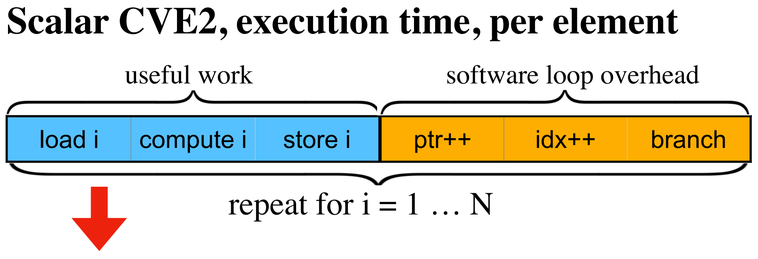
- Vector instructions are issued from decode to vec_unit.
- Scalar IF + ID/EX flow is preserved.
- vec_unit stalls ID/EX while sequencing over VL.
- Registered EX micro-op path improves timing.
- Scalar ALU/MUL and data memory port are reused.
- No SIMD lanes; one element is processed per cycle.



Why Speedup Without SIMD?

- Vector instructions amortize control over multiple elements.
- Hardware sequencing replaces scalar loop bookkeeping.
- Vector registers keep reusable operands close to compute.
- Unit-stride memory ops reduce repeated address work.

Scalar CVE2, execution time, per element



TinyRVV, execution time, per vector

i = 1 … VL	i = 1 … VL	i = 1 … VL
vle32.v	vector compute	vse32.v
Count i = 1 … VL	Count i = 1 … VL	Count i = 1 … VL
Read mem[addr]	Read VRF A[i]	Read VRF C[i]
addr++	Read VRF B[i]	Write mem[addr]
Write VRF A[i]	Write VRF C[i]	addr++

One element at a time under vector control. No SIMD lanes: speedup comes from amortized control and post-increment addressing.

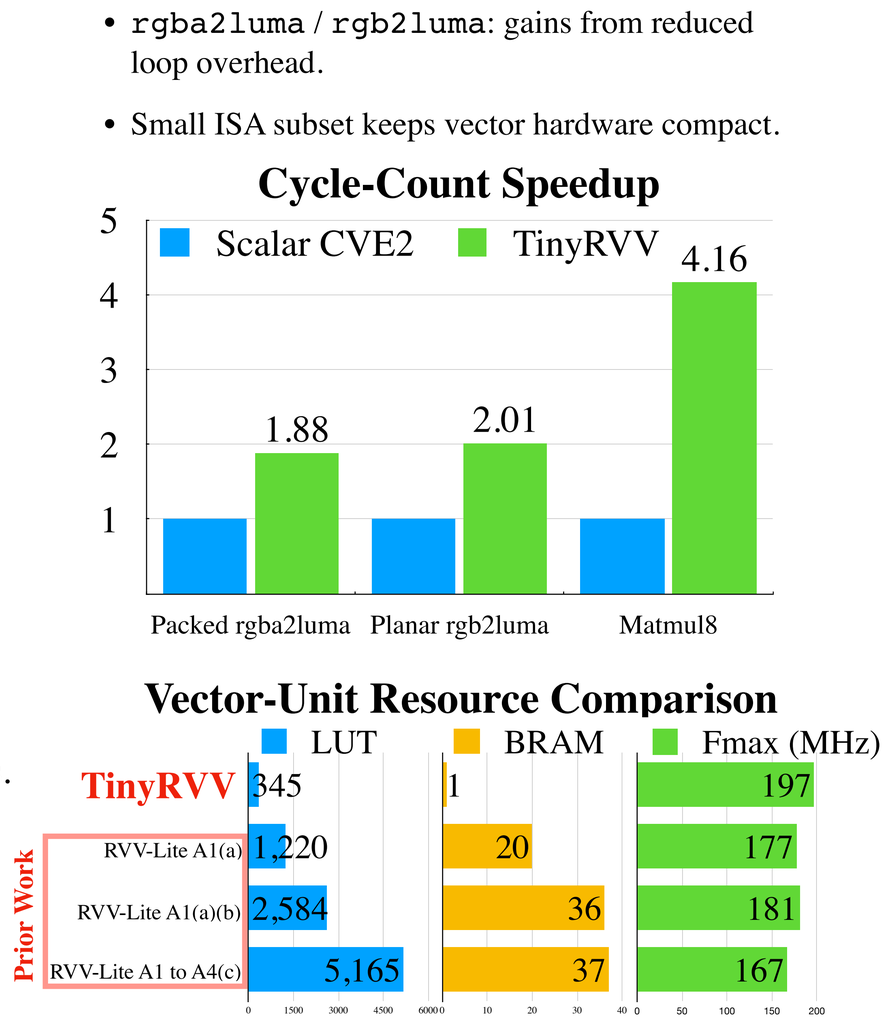
Results

Speedup with Limited Vector Hardware

- Cycle count improves across all kernels.
- `matmul8`: highest gain from data reuse.
- `rgba2luma` / `rgb2luma`: gains from reduced loop overhead.
- Small ISA subset keeps vector hardware compact.

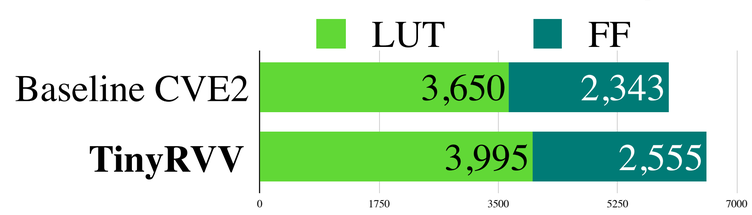
Cycle-Count Speedup

Vector-Unit Resource Comparison



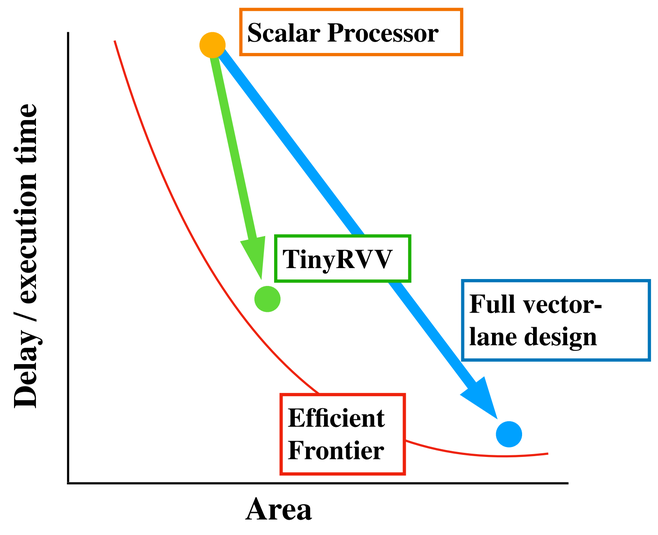
TinyRVV row shows vector extension logic only; scalar CVE2 ALU is reused.

CVE2 Core Area Before and After TinyRVV



CVE2 + TinyRVV adds 345 LUTs and 212 FFs over baseline.

Conceptual Delay–Area Tradeoff



Benchmark

Three kernels test different memory and reuse patterns.

Benchmark	What it tests
Packed rgba2luma	Packed pixel extraction
Planar rgb2luma	Multiple unit-stride loads
matmul8	Vector register reuse

Scalar: C with `-O2`; TinyRVV: hand-written vector assembly.

Link to GitHub Repository

https://github.com/UBC-ORCA/cve2-tinyrvv